Tipovi testiranja



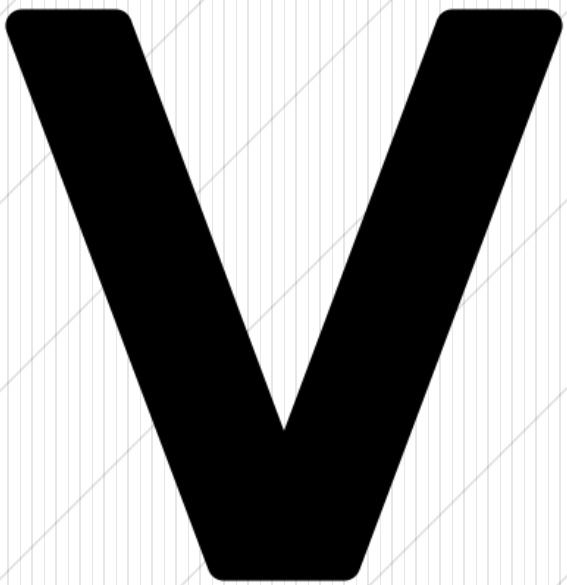
Autor: Goran Savić



Životinje se dele na one koje pripadaju caru, one koje su balzamovane, pse lutalice i one koje su nacrtane četkom od kamilje dlake.

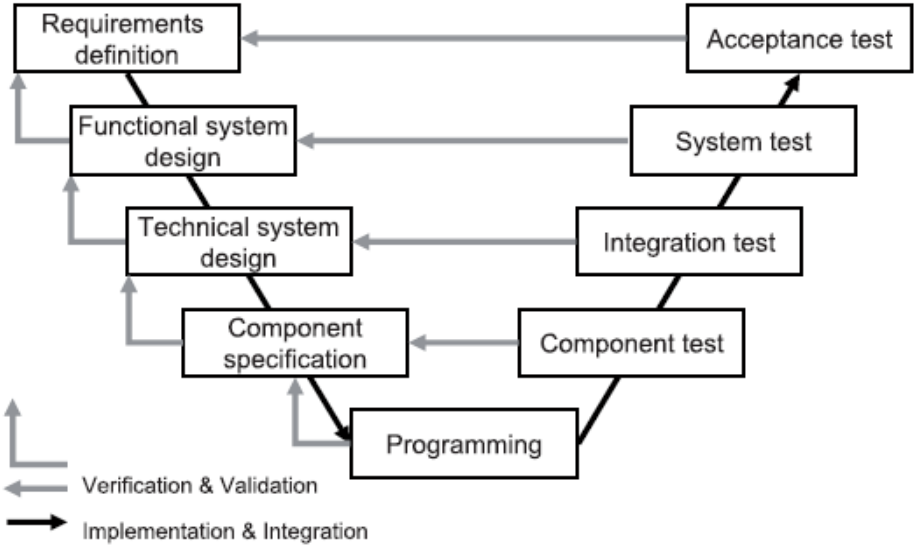
Jorge Luis Borges, „The Analytical Language of John Wilkins“

Testiranje u V-modelu razvoja softvera



Opšti V model

- Daje jednak značaj testiranju kao i razvoju softvera
- Leva strana predstavlja razvoj funkcionalnosti
- Desna strana predstavlja proces testiranja i integracije
  - komponente koje se razvijaju se integrišu u veće celine i njihova funkcionalnost se testira
  - svaka faza u testiranju verifikuje jednu od faza razvoja



V-model – aktivnosti vezane za razvoj

- Specifikacija zahteva
  - u ovoj fazi se specificiraju svrha i željene funkcionalnosti i karakteristike sistema
- Funkcionalni dizajn sistema
  - generalni dizajn sistema u skladu sa specifikacijom zahteva
- Tehnički dizajn sistema
  - arhitektura sistema
- Specifikacija komponenti
  - dizajn komponenti predviđenih arhitekturom
- Implementacija
  - implementacija koda u skladu sa prethodnim koracima

V-model – aktivnosti vezane za testiranje

- Testiranje komponenti
  - zove se i jedinično testiranje
  - nezavisno testiranje svake pojedinačne komponente da se utvrdi da li zadovoljava specifikaciju
- Integraciono testiranje
  - provera da li pojedinačne komponente sarađuju na specificirani način
- Sistemsko testiranje
  - verifikacija da li kompletan sistem zadovoljava funkcionalnosti kada se svi njegovi delovi integrišu u jednu celinu
- Test prihvatljivosti (eng. *acceptance test*)
  - provera da li sistem zadovoljava korisnikove zahteve i očekivanja

V-model testiranje

- Svaka faza testiranja verifikuje jednu od faza razvoja
- Svaka faza testiranja predstavlja jedan tip testiranja
  - ovde tipove klasifikujemo prema hijerarhijskom nivou u arhitekturi aplikacije na koji se testovi odnose
- Razvoj i testiranje se vrše paralelno
  - pripremne faze testiranja se vrše uporedo sa dizajnom softvera

Validacija i verifikacija

- Testiranjem se vrši
  - **validacija** softverskog proizvoda
    - utvrđuje se da li je realizovani proizvod pogodan za namenjenu svrhu
    - vodi računa o korisnikovom pogledu na aplikaciju
  - **verifikacija** softverskog proizvoda
    - na nivou pojedinačne faze utvrđuje da li je rezultat realizacije određene faze u skladu sa specifikacijom za tu fazu
    - više je fokusirana na tehničku realizaciju

Jedinično testiranje



Jedinično testiranje
Pojmovi



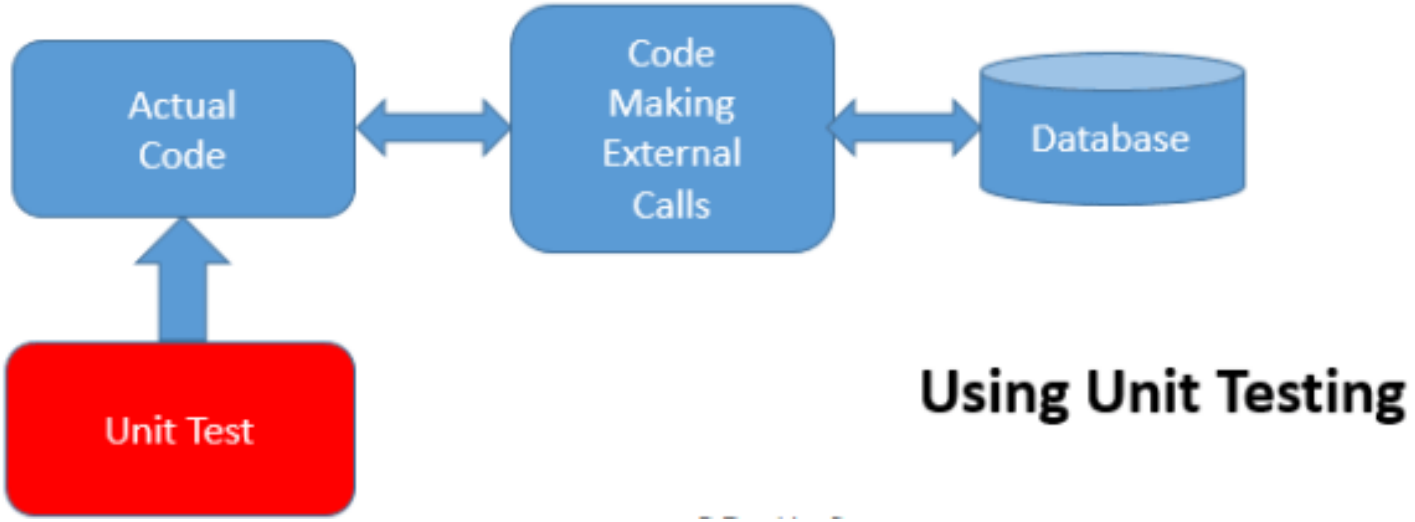
- Jedinično testiranje je testiranje pojedinačnih komponenti softvera
- Koriste se i nazivi testiranje komponenti, testiranje modula, testiranje klasa
- Treba da uporedi stvarnu i specificiranu funkcionalnost komponente
- Osim analize funkcionalnosti, može da uključi i analizu izvornog koda komponente

Jedinično testiranje
Test objekti

- Kod ovog tipa testiranja test objekti su pojedinačne softverske komponente
  - klasa najčešće
  - može da bude i uskladištena procedura, skript za bazu, ...
- Glavna karakteristika je da se softverska komponenta testira nezavisno i izolovano od ostatka sistema
  - na ovaj način smo sigurni da verifikujemo ispravnost rada pojedinačne komponente bez uticaja programskog koda u drugim komponentama
  - detektovan problem onda definitivno ukazuje na nedostatak u testiranoj komponenti

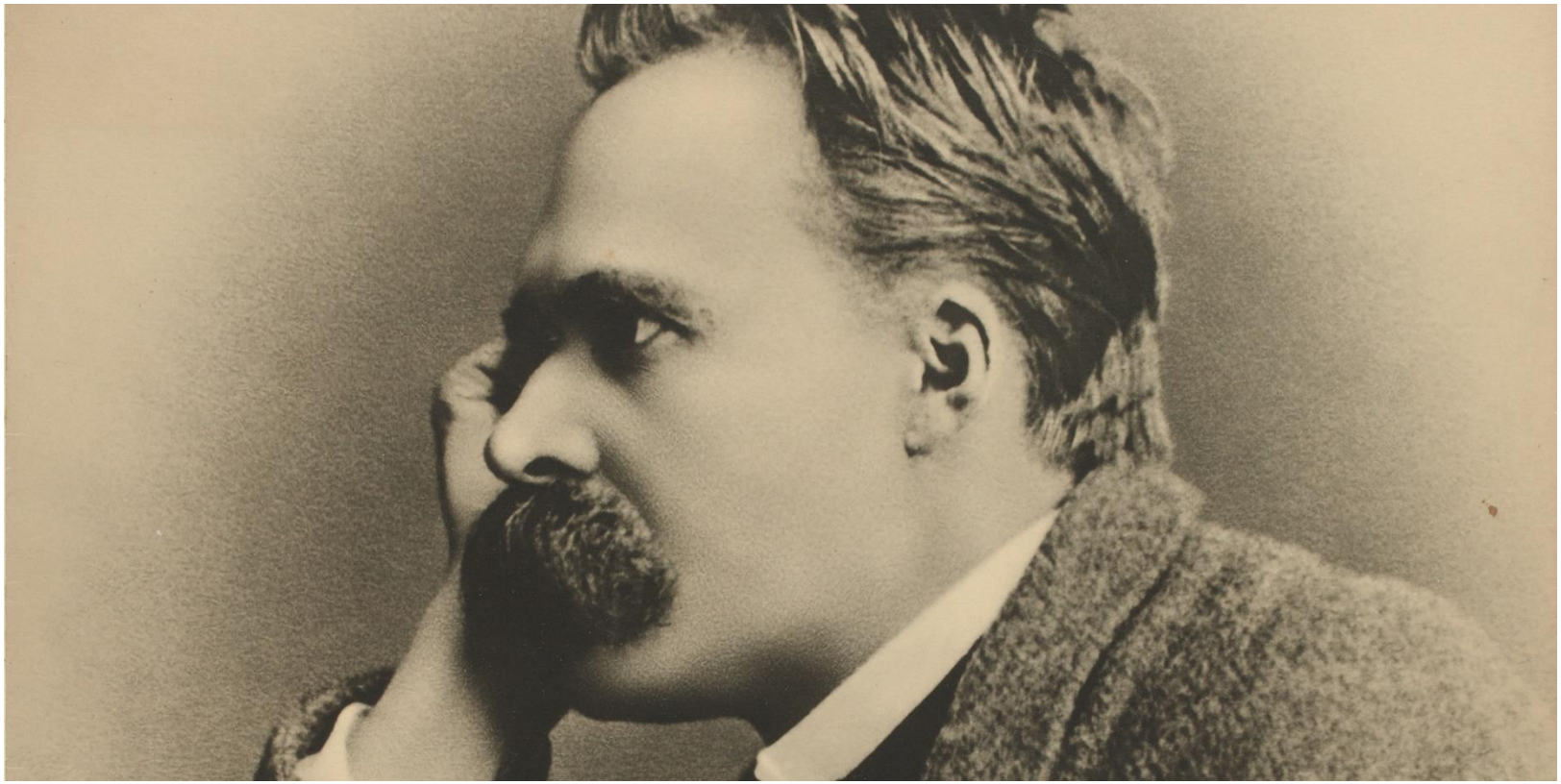
Jedinično testiranje
Izolovano testiranje

- Kako testirati klasu u izolaciji kada njeno ponašanje zavisi od koda u drugim klasama?
- Npr. klasa ima referencu na drugu klasu
  - U toku izvršavanja svoje metode poziva metodu druge klase
  - Ispravnost izvršavanja zavisi i od koda u drugoj klasi



* preuzeto sa www.dotnetcurry.com

Jedinično testiranje
Izolovano testiranje

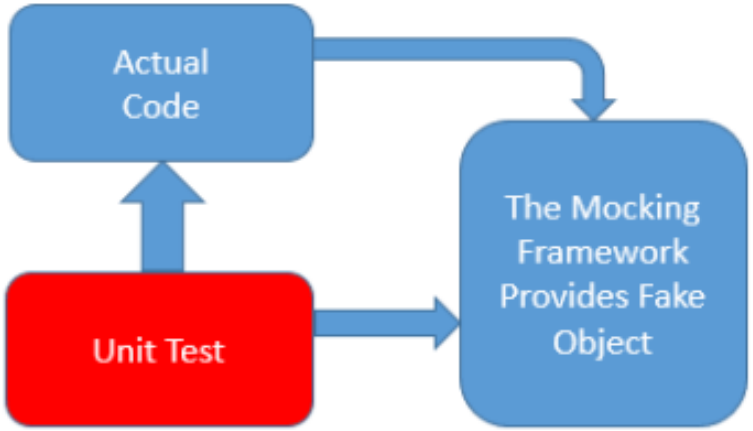


To je moja prva ljudska pamet što dozvoljavam da me prevare da ne bih bio na oprezu pred varalicama.
(Tako je govorio Zaratustra)

Friedrich Nietzsche
(1844 – 1900)

Jedinično testiranje
Test dvojnik (*test double*)

- Umesto pravih objekata koje klasa referencira postavljaju se objekti dvojnici koji pojednostavljuju ili simuliraju ponašanje referenciranih objekata
- Ovi objekti omogućuju da se test izvrši, a da bude fokusiran samo na objekat koji testira

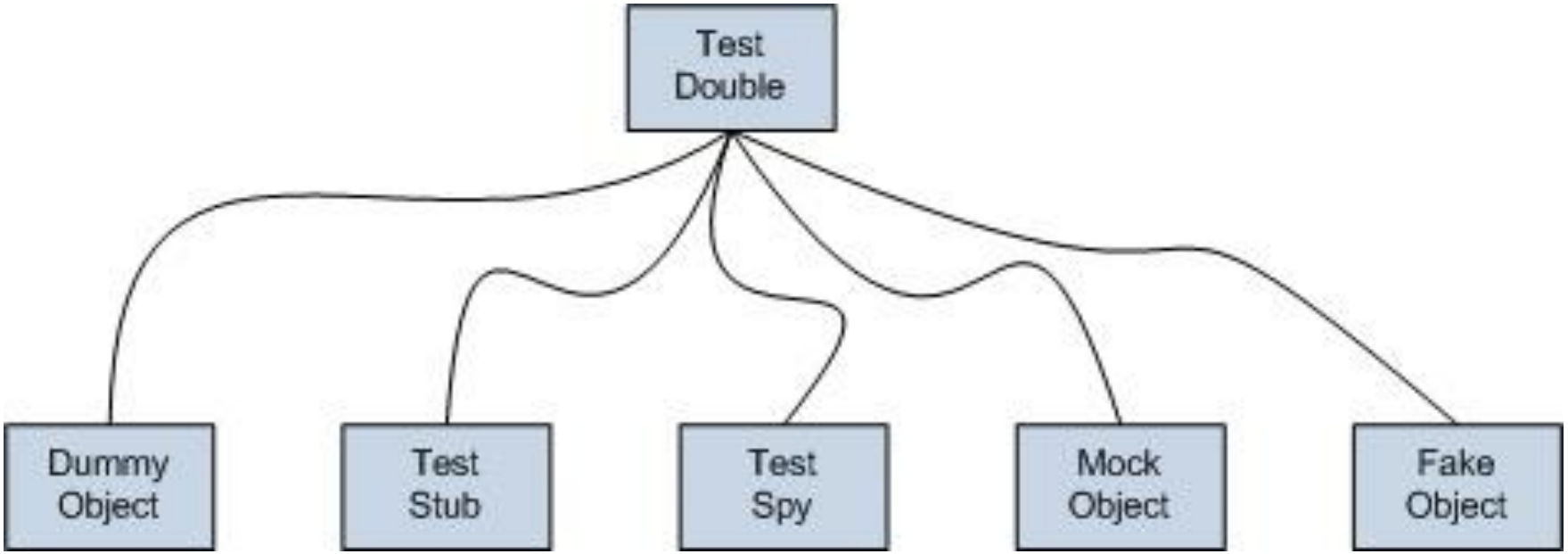


Using Mocking Framework

* preuzeto sa www.dotnetcurry.com

Jedinično testiranje
Test dvojnik (*test double*)

- Postoje različiti tipovi dvojnika



Jedinično testiranje
Tipovi test dvojnika

- *Dummy* objekat
  - objekat koji mora postojati u kodu, ali se nikad ne koristi
  - najčešće postoji da popuni obaveznu listu parametara
  - termin se koristi i za svaki objekat inicijalizovan podacima čiji sadržaj nije važan za test
    - npr. ako testiramo kreiranje nekog entiteta, sami podaci koje entitet sadrži nam nisu važni

Jedinično testiranje
Tipovi test dvojnika

- Lažni (*fake*) objekat
  - sadrži stvarni programski kod koji se izvršava
  - jedino je zbog jednostavnosti ili brzine implementacija drugačija u odnosu na aplikaciju u produkciji i prilagođena je testiranju
  - dobar primer je korišćenje *in-memory* baze podataka za potrebe testiranja

Jedinično testiranje
Tipovi test dvojnika

- *Stub* objekat
  - simulira izlaze stvarnog objekta
  - na predefinisane ulaze daje predefinisane izlaze
  - zna da reaguje samo na ulaze koje konkretan test predviđa

Jedinično testiranje
Tipovi test dvojnika

- *Mock* objekat
  - simulira ponašanje stvarnog objekta
    - za razliku od *stub* objekta koji simulira samo izlaze
  - Ako stvarni objekat sadrži sekvencu poziva određenih metoda, *mock* objekat ponavlja ovu sekvencu
    - na ovaj način dobijamo dvojnika koji se i ponaša kao stvarni objekat
    - sami izlazi koji su rezultat ponašanja su i dalje simulirani

Jedinično testiranje
Tipovi test dvojnika

- *Spy* objekat
  - modifikovani stvarni objekat
  - deo ponašanja je stvarno ponašanje
  - deo ponašanja je simuliran zbog potreba testiranja
  - korišćenjem *spy* objekata test može da dobija stvarno ponašanje jednog dela, a simulirano ponašanje drugog dela funkcionalnosti

Jedinično testiranje
Način testiranja

- Svodi se na pisanje programskog koda koji
  1. poziva funkciju čiju funkcionalnost proveravamo i prosleđuje joj određene ulazne parametre
  2. proverava da li je rezultat funkcije očekivan za prosleđene ulazne parametre
     - rezultat može biti povratna vrednost funkcije ili
     - neki drugi efekt koji funkcija proizvodi (npr. pojava fajla na disku)

Jedinično testiranje
Autor testova

- Za pisanje jediničnih testova programski kod testirane jedinice mora biti dostupan
- Potrebno je poznavati svrhu i interfejs svake funkcije koja se testira
- Iz ovih razloga jedinične testove najčešće pišu programeri

Jedinično testiranje
Ciljevi testiranja

- Glavni cilj je testiranje funkcionalnosti komponente
  - kakve izlaze daje za zadate ulaze
- Cilj se ostvaruje nizom test slučajeva od kojih svaki testira određenu kombinaciju ulaza/izlaza
- Jedan deo test slučajeva moraju biti i negativni testovi koji proveravaju kako komponenta reaguje na nedozvoljene ili nepredviđene ulaze

Jedinično testiranje
Test strategija

- Strategija za jedinično testiranje najčešće podrazumeva uključivanje znanja o strukturi koda
  - time ono spada u *white-box* testiranje
  - test slučajevi bi trebali biti napisani tako da pokriju različite putanje u programskom kodu
- Nekada su jedinični testovi *black-box* testiranje jer ne uzimaju u obzir strukturu testirane jedinice
  - ako test piše osoba koja nije programirala ili ne poznaje testiranu komponentu
  - ako je programski kod testa programski generisan
  - ako se testovi pišu pre implementacije komponenti, kao kod *test-driven development* metodologije razvoja
  - ...

Integraciono testiranje



Integraciono testiranje
Pojmovi



- Integraciono testiranje je testiranje namenjeno otkrivanju nedostataka u interfejsima i interakciji između međusobno povezanih softverskih komponenti
- Primer
  - serverska komponenta uspešno isporučuje JSON podatke
  - klijentska komponenta uspešno konzumira XML podatke
  - jedinični testovi za obe komponente se uspešno izvršavaju
  - integracioni test treba da utvrdi otkaz sistema jer formati kojima pojedinačne komponente komuniciraju nisu usklađeni

Integraciono testiranje
Test objekti

- Kako se pojedinačne komponente razvijaju, tako se integrišu u veće celine
- Trebalo bi da postoji integracioni test pri svakoj integraciji komponenti
  - na ovaj način verifikujemo ispravnost integrisane celine
  - ona onda može biti osnov za dalju integraciju sa drugim celinama
- Neke od komponenti koje se integrišu mogu biti „strane“ komponente
  - npr. drugi sistem sa kojim naš sistem komunicira
  - ti delovi nisu pod našom kontrolom i mogu da izazovu nepredviđene otkaze

Integraciono testiranje
Način testiranja

- Svodi se na izvršavanje koda u kojem komponente koje se integrišu međusobno komuniciraju
- Mogu se koristiti dodatni alati za praćenje komunikacije između komponenti
  - npr. *browser* sadrži alat za prikaz podataka koji se razmenjuju koristeći HTTP protokol

Integraciono testiranje
Ciljevi testiranja

- Cilj je pronaći nedostatke u komunikaciji i konflikte između komponenti koje se integrišu
- Ovim se smanjuju troškovi otkaza time što će se
  - nedostaci u integraciji pronaći što ranije
    - ako se integracioni test sprovede čim se komponente integrišu, integrisana celina se kao verifikovana dalje integriše u sistem
  - nedostaci lakše locirati
    - pouzdano se zna da se nedostatak nalazi u komunikaciji komponenti čija se integracija testira

Integraciono testiranje
Strategije integracije

- Strategija određuje kojim redosledom se komponente trebaju integrisati kako bi integracioni testovi bili efikasni
- Što se ranije realizuju integracioni testovi
  - u ranoj fazi se pronalazi nedostaci, ali
  - zahteva se simuliranje svih komponenti koje još nisu implementirane
- Strategija koja će biti izabrana zavisi od
  - arhitekture sistema
  - plana projekta
  - plana testiranja

Integraciono testiranje
Strategije integracije

- Integracija odozgo na dole
  - Integracija kreće sa komponentama na vrhu hijerarhije
  - Komponente na nižem nivou hijerarhije koje još nisu implementirane se simuliraju
  - Kako se niže komponente implementiraju tako se verifikuje njihov rad stavljanjem u test umesto simuliranih objekata

Integraciono testiranje
Strategije integracije

- Integracija odozdo na gore
  - Testiranje počinje integracijom osnovnih komponenti na najnižim nivoima hijerarhije
  - Time se formiraju integrisane komponente na višim nivoima hijerarhije koje se dalje integrišu i tako redom
  - Testovi ne moraju da simuliraju ponašanje komponenti na nižim nivoima jer će one već biti implementirane
  - Pozivanje nižih komponenti mora biti realizovano iz posebnog test okruženja jer nisu implementirane komponente na višim nivoima koje te pozive obavljaju

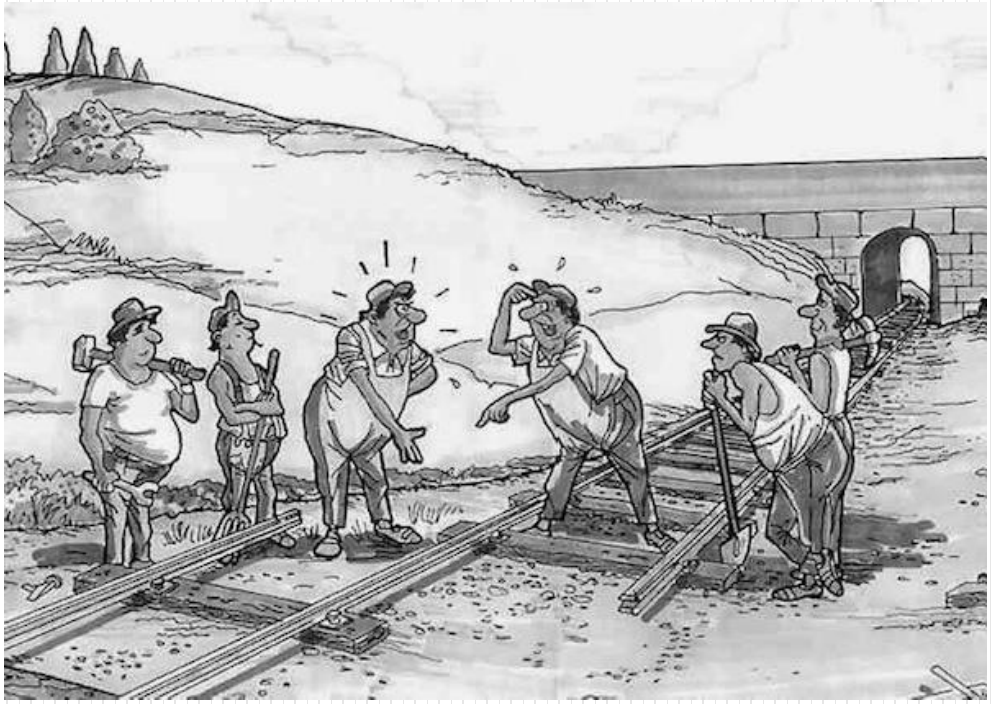
Integraciono testiranje
Strategije integracije

- Ad hoc integracija
  - Komponente se integrišu redom kojim se završava njihova implementacija
  - Prednost je rana integracija i pronalaženje nedostataka
  - Mana je potreba za simuliranim komponentama i pozivanjem komponenti iz specijalnog test okruženja
    - nema garancije da postoje implementirane kako komponente na višim tako ni one na nižim nivoima hijerarhije

Integraciono testiranje
Strategije integracije

- *Big bang* integracija
  - integraciono testiranje se vrši tek kada se sve komponente razviju i integrišu
  - odjednom se verifikuje međusobna povezanost svih komponenti
    - svi otkazi će se pojaviti istovremeno i biće teško lokalizovati nedostatke
  - generalno treba izbegavati neinkrementalne strategije integracije
    - potrebno je integraciju vršiti uklapanjem komponenti jedna-po-jedna, a ne sve odjednom

Sistemsko testiranje



Sistemsko testiranje
Pojmovi



- Sistemsko testiranje je testiranje kompletno integrisanog sistema da bi se utvrdilo da li sistem u celini ispunjava zahteve
- U ovom značenju, sinonim je za testiranje „s kraja na kraj“ (eng. *end-to-end testing*) (e2e)
  - Kod e2e testiranja, naglasak je na akciji od početka do kraja
  - npr. od korisnikovog unosa do perzistencije u bazu podataka

Sistemsko testiranje
Pojmovi

- Vrši se jer niži nivoi testiranja verifikuju aplikaciju samo sa tehničkog aspekta, a sistemsko testiranje vrši verifikaciju sa korisnikovog aspekta
  - tek kada je ceo sistem integrisan moguće je izvršiti kompletne korisničke akcije i verifikovati njihovu ispravnost

Sistemsko testiranje
Pojmovi

- Termin može i da se odnosi na testiranje celog sistema u produkcionom okruženju
  - test okruženje treba što više da liči na okruženje u kojem će sistem biti korišćen
  - ovo se odnosi i na softversko i na hardversko okruženje
  - vrlo često se okruženje u kojem se sistem razvija razlikuje od onog u kojem će biti korišćeno
    - često je pretpostavka o sličnosti ova dva okruženja uzrok velikog broja otkaza u produkciji



Kući mi je to radilo.

Sistemsko testiranje
Ciljevi testiranja

- Cilj je testirati funkcionalne i nefunkcionalne zahteve sistema u celini
- Treba utvrditi u kojim delovima sistem netačno, nepotpuno ili nekonzistentno zadovoljava zahteve
- Takođe, treba utvrditi i zahteve koji su neevidentirani ili zaboravljeni u specifikaciji

Test prihvatljivosti



Jedinice vs jedinstvo



Rene Guenon
(1886 – 1951)

Sa jedne strane imamo „jedinice“, sa druge „jedinstvo“. Mnoštvo nižeg reda (jedinice) je po definiciji čisto kvantitativno (...) Mnoštvo višeg reda (jedinstvo) je kvalitativno.
(Vladavina kvantiteta i znaci vremena)

Acceptance test
Pojmovi



- Test prihvatljivosti je test kojim se utvrđuje da li sistem zadovoljava potrebe, zahteve i očekivanja naručioca i koji ima za cilj da naručilac verifikuje da li prihvata da preuzme sistem na korišćenje
- Ovo je test koji nije samo odgovornost proizvođača softvera
  - u ovaj test se obavezno uključuje i naručilac
- Ovaj test se realizuje u različitim fazama kreiranja softvera
  - softverski proizvodi razvijeni za široko tržište
    - verifikuju se od strane korisnika kada izvrši instalaciju i krene korišćenje
    - npr. Microsoft Office
  - za unapred poznate korisnike test prihvatljivosti može i ranije
    - kroz verifikaciju prototipa ili neke od komponenti aplikacije

Acceptance test
Tipovi

- Postoje sledeći tipovi testa prihvatljivosti
  - Ugovorni
  - Korisnički
  - Operativni
  - Specifičan za okruženje

Acceptance test
Ugovorni test (*Contract Acceptance Test*)

- Koristi se kod razvoja softvera za unapred poznatog naručioca
- Postoji ugovor u kojem su definisani zahtevi sistema
- Testiranjem se verifikuje i označava ispunjenost svakog od zahteva
- Proizvođač bi u okviru sistemskog testiranja već trebao da proveri ispunjenost svakog zahteva
- Test prihvatljivosti se onda svodi na ponovno pokretanje sistemskog testa u okruženju naručioca
  - ne mora da znači da će testovi biti uspešni i u novom okruženju

Acceptance test
Korisnički test (User Acceptance Test)

- Koristi se kada naručilac i korisnik nisu isti
- U ovom testu krajnji korisnik verifikuje da li je sistem upotrebljiv
- Ovo je vrlo često najteži test jer
  - krajnji korisnik ima najmanju sposobnost i interes da svoja očekivanja i zahteve izrazi i formalizuje
  - bez prihvatanja od strane krajnjeg korisnika sistem će teško biti uveden u upotrebu
  - ovaj test se obično vrši u kasnoj fazi razvoja kada je svaka izmena komplikovana i skupa

Acceptance test
Operativni test (Operational Acceptance Test)

- Obezbeđuje prihvatanje softvera od strane sistem administratora
- Oni verifikuju tehničke aspekte sistema
  - skladištenje podataka
  - oporavak sistema
  - upravljanje resursima
  - upravljanje korisnicima
  - bezbednost sistema
  - hardverske zahteve
  - uticaj na postojeće softversko okruženje

Acceptance test
Test specifičan za okruženje (field testing)

- Koristi se kada je softver namenjen za upotrebu u raznorodnim okruženjima
- Tada je praktično nemoguće izvršiti sistemski test u svim mogućim okruženjima



- Test specifičan za okruženje je test preliminarne verzije softvera koji se vrši od strane određene (reprezentativne) grupe korisnika kako bi se utvrdilo ponašanje softvera u korisnikovom okruženju koje je bilo nepoznato ili nespecificirano u toku razvoja softvera
- Svakako mora da mu prethodi sistemski test

Acceptance test
Test specifičan za okruženje (field testing)

- Ovaj test je posebno važan za proizvode namenjene širokom tržištu
- Grupa korisnika koja će testirati softver može biti
  - unapred odabran raznorodan uzorak koji dobro reprezentuje buduće tržište
  - proizvoljna grupa korisnika koji odluče da preuzmu javno dostupnu probnu (*trial*) verziju proizvoda
- Razlikujemo dva tipa ovakvih testova
  - alfa testiranje
    - organizuju se na lokaciji proizvođača pod njegovim upravljanjem
  - beta testiranje
    - vrši ih korisnik u okviru svojih resursa i okruženja

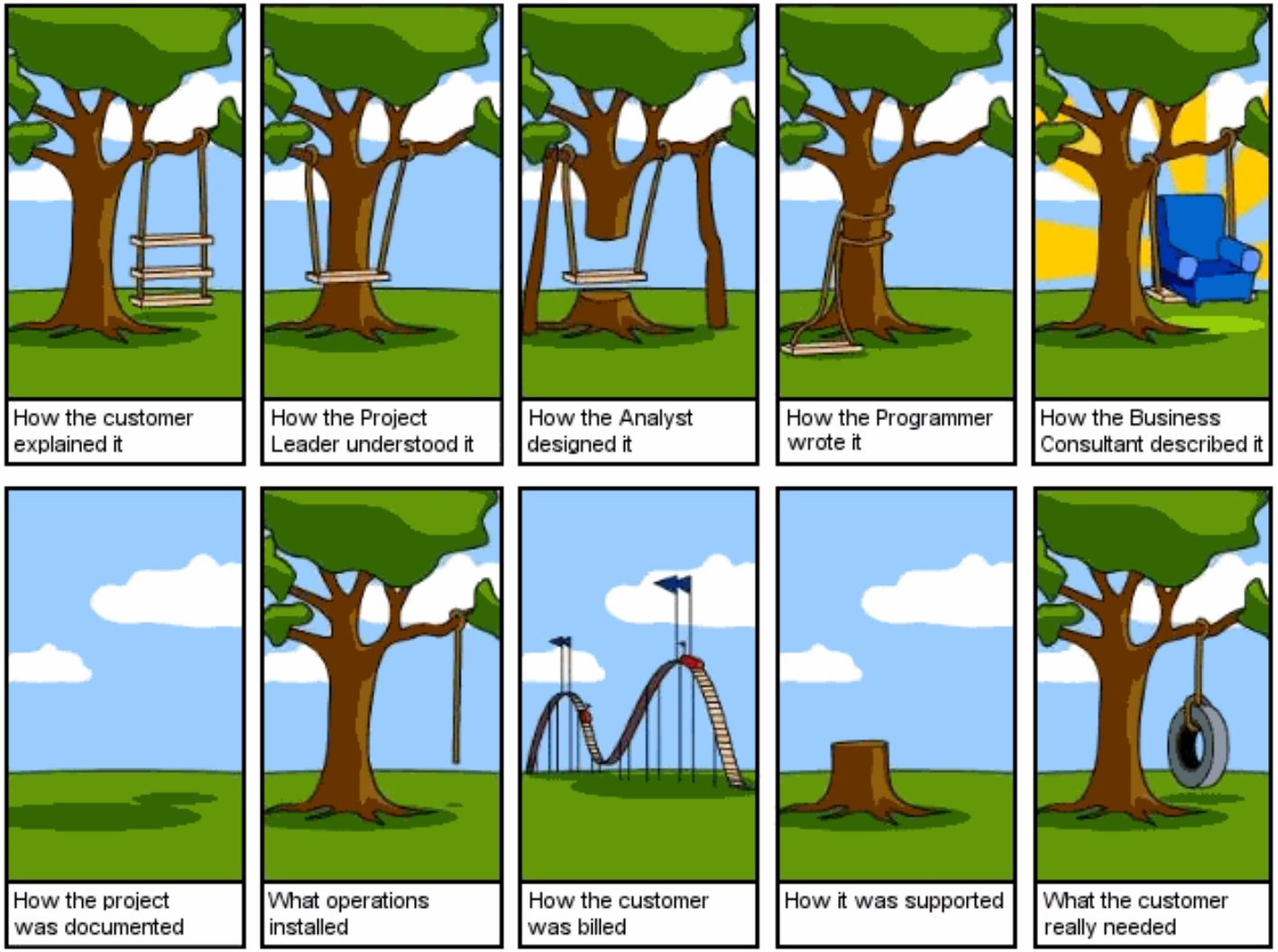
Acceptance test
Test specifičan za okruženje (field testing)

- Još jedna varijanta testiranja proizvoda za tržište je *dogfood* test
  - proizvod se najpre koristi u okviru kompanije koja ga razvija
  - ideja je da će softver biti kvalitetniji kada oni koji ga razvijaju direktno osete posledice njegovih nedostataka
  - „ako proizvodiš hranu za pse, najpre je ti probaj“
- Google Nexus telefoni su ulazili u upotrebu po ovom principu
- Microsoft ovako testirao rane verzije Windows OS
- Apple je promovisao zamenu pisaćih mašina računarima tako što je najpre u svojoj kompaniji uveo u potpunosti ovu zamenu

Acceptance test
Problemi

- U većini sistema zahtevi su neprecizno ili nepotpuno specificirani
  - teško je testirati sistem za koji nismo sigurni kako treba da se ponaša
- Korisnik umesto zahteva češće ima očekivanja od sistema
  - formalizacija očekivanja u zahteve je obično kamen spoticanja u razvoju i kasnijem testiranju
  - korisnik obično shvati kako funkcionalnost treba da izgleda i radi tek kada je implementirana drugačije
    - ispostavlja se da je zahtev loše postavljen
  - zato je sistemsko testiranje umesto kraj razvoja često samo početak nove iteracije revidiranja zahteva, pa usklađivanja aplikacije prema novim zahtevima

Acceptance test Problemi





Softverski sistem mora biti kontinuirano prilagođavan ili postaje progresivno sve manje zadovoljavajući.

(Lemanov zakon evolucije softvera)

Manny Lehman
(1925 - 2010)

Testiranje nove verzije softvera

- Postavljanje aplikacije u produkciju je samo početak nove faze u životnom ciklusu proizvoda
- Održavanje softvera podrazumeva nove izmene
  - ispravljanje uočenih nedostataka
  - prilagođavanje softvera novom okruženju
  - implementacija novih zahteva
- Pri svakoj od izmena potrebno je ponovo obaviti kompletan ciklus testiranja
- Nove verzije mogu biti
  - neplanirane, izazvane ispravkama nedostataka (*hot fixes*)
  - planirano izdavanje nove verzije
- Prestanak korišćenja softvera zahteva testiranje arhiviranja i migracije podataka

Regresiono testiranje

- Kada se izvrši izmena potrebno je testirati ispravnost unešene izmene
- Zbog mogućih bočnih efekata izmene, potrebno je i ponovo testirati ranije razvijene funkcionalnosti



- Regresiono testiranje je testiranje već uspešno testiranog dela softvera nakon što su u drugim delovima softvera izvršene izmene, kako bismo verifikovali da unešene izmene nisu negativno uticale na postojeće ispravne funkcionalnosti
  - odnosi se na sve nivoe i tipove testiranja

Funkcionalno vs nefunkcionalno testiranje

- Funkcionalno testiranje se odnosi na testiranje funkcionalnih zahteva
  - ovi zahtevi specificiraju ponašanje sistema
  - šta sistem može da uradi
  - bez ispunjavanja ovih zahteva, sistem nema smisla
- Testiranje zasnovano na zahtevima
  - uzima formalno evidentirane zahteve kao polaznu osnovu
  - za svaki zahtev se prave test slučajevi koji verifikuju da li je zahtev ispunjen
- Testiranje zasnovano na poslovnom procesu
  - verifikuje test scenario kreiran na osnovu sekvence korisničkih akcija

Funkcionalno vs nefunkcionalno testiranje

- Nefunkcionalno testiranje se odnosi na testiranje nefunkcionalnih zahteva
  - ovi zahtevi specificiraju kriterijum kojim se meri kvalitet implementacije funkcionalnih zahteva
  - npr. performanse, upotrebljivost, portabilnost, lakoća održavanja
- Postoje različiti tipovi nefunkcionalnog testiranja
  - u zavisnosti od toga koji nefunkcionalni zahtev testiramo

Tipovi nefunkcionalnog testiranja



- Test opterećenja (*load test*)
  - merenje ponašanja sistema u uslovima povećanog opterećenja (npr. broja korisnika ili količine pozvanih operacija) kako bi se utvrdilo koji stepen opterećenja sistem može da izdrži



- Test performanse (*performance test*)
  - merenje performanse softvera (npr. brzina obrade ili vreme odziva) za određenu fukcionalnost (obično se meri zavisno od opterećenja)



- Test velike količine podataka (*volume test*)
  - testiranje ponašanja sistema kada upravlja velikom količinom podataka

Tipovi nefunkcionalnog testiranja

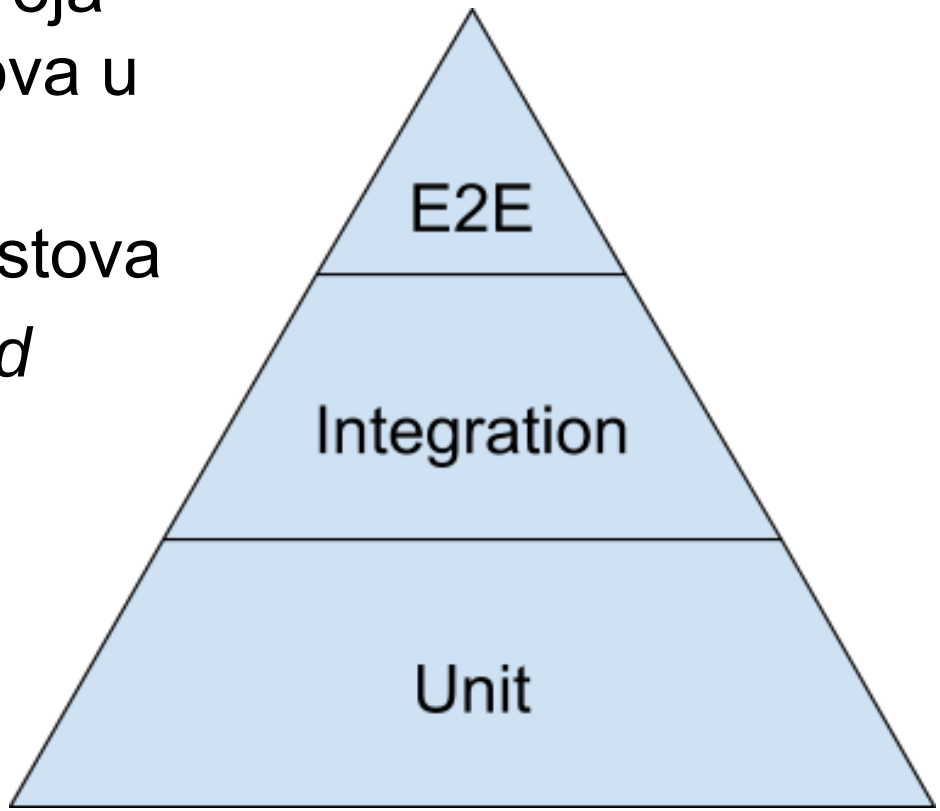
- Stres test (*stress test*)
  - Testiranje ponašanja sistema u uslovima preopterećenja iznad normalnih granica
- Test bezbednosti
  - Testiranje bezbednosnih aspekata sistema kao što su autorizacija pristupa, hakerski napadi, ...
  - podtip je *penetration testing*
    - testiranje bezbednosti tako što korisnik pokušava da nađe „rupu“ u sistemu i ugrozi rad sistema ili pristupi podacima i funkcionalnostima koji mu nisu dozvoljeni
- Test robustnosti (*robustness test*)
  - testiranje ponašanja sistema u uslovima za koje nije dizajniran
  - nedozvoljeni ulazi, greške u korišćenju, hardverski otkazi, ...

Piramida testiranja



Piramida testiranja

- Mike Cohn, 2009.
- Piramida ukazuje na međusobni odnos broja različitih tipova testova u sistemu
- Najviše jediničnih testova
- Najmanje *end-to-end* testova



Piramida testiranja – argumenti ZA

- Argumenti za kreiranje najviše jediničnih testova
  - jedinični testovi testiraju izolovano ponašanje što olakšava lociranje nedostatka u slučaju neuspešnog testa
    - kod e2e testa je teško lociranje problema
    - utvrdi se da sistem ne radi ali se ne zna zašto
    - pošto e2e test testira kompletan sistem, uzrok može biti bilo gde (hardver, softvrsko okruženje, neki kod u aplikaciji, ...)
  - jedinični testovi su brzi za izvršavanje
  - e2e testovi su zahtevni za održavanje
    - obzirom da simuliraju korisničku interakciju, neznatna promena grafičkog interfejsa zahteva promenu testa

Piramida testiranja – argumenti PROTIV

- U praksi se često ne poštuje odnos testova predviđen klasičnom piramidom testiranja
  - kreira se najviše e2e testova
- Argumenti PROTIV klasične piramide
  - jedino e2e testovi testiraju sistem u celini
    - za razliku od jediničnih testova koji testiraju izolovano ponašanje jednog dela sistema
  - e2e testovi simuliraju stvarno korišćenje sistema
    - simuliraju korisnikovu interakciju sa sistemom
  - e2e testovi testiraju kompletno okruženje
    - različite browsere, upotrebu hardvera i sl.
  - sporost izvršavanja e2e testova se može ublažiti paralelizacijom izvršavanja testova i boljim hardverom

Sladoled anti-patern

- Pomenuti argumenti često dovode do obrnute piramide testiranja
  - poznata kao sladoled anti-patern (*ice-cream cone anti-pattern*)

